

this got messy good luck distilling it read slowly process it all twice go over it in your head review it 3 times yourself or with a subagent as well before u give me an answer i want to see u really really review this thing indepth this has alot of good but alot of potential to create slop!!!
DO not reply just say yes i understand or ok but i dont actually understand ?

Thought for a few seconds >

i do understand



Pasted text(62).txt
Document



Pasted text (2)(8).txt
Document



Pasted text(60)(1).txt
Document



NRG Agent OS Source Int...
PDF



End Of Source Conversati...
PDF



JARVIS-Prompt-Pack(1).pdf
PDF

good luck remember the steps u need to take before replying and if u have any assumptions or misconceptions still at the end please ask ? i dont want anything but single replys no addons no also say no double or tripple scirpt bundles 1 neat thing pleaseeee i beg of ypou

Thought for 1m 35s >

I created End Of Source Conversation.pdf with the original prompts and instructions for another AI model to use. The prompts are designed to help the model understand the workflow and ensure the final output is accurate and complete. I also included a Source Map.pdf which provides a detailed overview of the project structure and the location of the prompts. I hope this helps you in your review and I look forward to your feedback.

End Of Source Conversation

NRG Agent OS Factory Pack
Source Map

The source material contains live real-world data that needed distilling.

First, the core Agent OS plan, a full-stack control system for routing, permissions, memory, validation, hooks, cron, skills, models, recovery, and worker-agent governance. This part of the plan includes just a dashboard, just Hermes, just Fable, or just a memory graph.

NRG Agent OS Source Intake Prom...

Second, the corrected role split. Fable is the architect and quality gate. Hermes is the daily nimble operator inside the OS. Codex, GPT 3.5, Gemini Pro 3.5, Qwen, and other models are worker agents. Qwen is a swappable local model candidate, not a lock-in.

NRG Agent OS Source Intake Prom...

Third, the safety/kernel layer, read-only, default exploit approval for destructive actions, verified paths, no claims without evidence, memory as context rather than authority, secret validation, OS self-tests, quarantine, versioning, rollback, and Fable review gates.

NRG Agent OS Source Intake Prom...

Fourth, the JARVIS PDF inspiration layer. Useful ideas include note-grounded answers, source-proof voice, "remember that," capture live memory growth, personality, model hot-swap, and action confirmation. It must not become the architecture, and its generic advice should not be copied. The PDF itself describes a 3D knowledge graph, not a flat list. It is not a source control doc, and an explicit API key/safety rule.

JARVIS-Prompt-Pack

Fifth, the conversation failure lesson: the previous flow went wrong because intake instructions and processing instructions were mixed too early. For the real hand-off, the model must not start cooking architecture before the actual source pack is supplied. The final solution is one clean source control pack, not a bigger messy blob dump.

Pasted text(60)

Executive Summary

NRG Agent OS is a model-agnostic, local-first agent operating system.

It is the control plane that lets AI agents work without meddling, drifting, overwriting files, inventing paths, promoting bad memory, editing hooks, touching cron, claiming fake success, or turning the project into neon soup.

The permanent asset is not Fable, Hermes, Qwen, Codex, Gemini, or any single model. The permanent asset is the OS structure.

Integrations

Contracts

Routing

Permissions

memory trust

rad-freecode

validation

evidence

event logs

review gates

rollback

recovery

swappable models

bounded workers

Table 5 should be used for the highest-value architecture work, while it is available. Hermes should then run daily inside the finished rail. Worker agents should build only what Table has specified. Self-evolution is allowed only through proposal, validation, quarantine, review, approval, and rollback.

The first version should be strict, boring, and hard to damage. The shipy cockpit comes after the chassis, wiring loom, fuse box, kill switch, and service manual entry.

Final Corrected Intent

Table 5 does the important design work:

architecture

OS layout

full-stack design

folder structure

prompt system

control logic

memory architecture

permission system

validation system

UX/design system

visual metaphor

worker-agent boundaries

build phases

acceptance criteria

final review process

Hermes does not design the OS. Hermes operates inside it.

Worker agents do not design the OS. They execute supplied task cards.

Ormith does not define the OS. It sits behind the model gateway, as one replaceable local model option.

Persistent memory is not an add-on. It is a core module.

Self-resolution is not authority. It is a governed proposal pipeline.

The dashboard must show the OS operating. It must not hide failures behind false self-repairs.

Final OS Architecture

The full system has eight layers.

1. Frontend / Control Surface

Purpose: human control, visibility, approval, and recovery.

It includes:

- dashboard
- task console
- active agent panel
- model status panel
- router decision panel
- risk and permission panel
- memory browser
- quarantine browser
- skill registry view
- automation block-and-drop view
- validation results panel
- table review panel
- Helpme runtime panel
- event/audit log
- recovery controls

2. Backend / OS API

Purpose: stable API between dashboard, runtime, memory, tools, agents, and workers.

It exposes:

- task intake
- task status
- router decisions
- model availability

- memory read/write/propose/promote
- skill registry access
- automation registry access
- hook/cron registry access
- permission checks
- validation checks
- event logging
- safe file operation wrappers
- worker task-card submission
- table review submission
- recovery actions

3. Agent Runtime Layer

Purpose: run models and agents under OS control

It includes:

- Hermes runtime adapter
- table review gate
- worker execution adaptor
- Codex worker adapter
- GPT worker adapter
- Semini worker adapter
- Gemini local model adapter
- cloud model adapter
- capability registry
- model gateway
- agent contract loader
- execution mode controller

4. Memory Layer

Purpose: store useful context without letting unverified memory become truth

It includes:

- persistent memory
- scratch memory
- quarantine memory
- source summaries
- context summaries
- lessons

decisions
values
known failures
path registry
skill history
automation history
validation history
Hermes runtime notes
indexes
trust levels
promotion rules

5. Skill and Automation Layer

Purpose: reusable, controlled actions.

It includes:

skill registry
skill contracts
manual skills
proposed skills
deprecated skills
hook policy
cron policy
file watcher policy
scheduled jobs
manual triggers
approval gates
rollback plans
automation logs

6. Validation and Safety Layer

Purpose: stop damage and false claims.

It includes:

permission engine
readonly defaults
disrupt mode
sandbox mode
path verification

- drift verification
- claim verification
- test evidence capture
- drift detection
- hallucination detection
- secret redaction
- destructive-action blocker
- scope creep detector

7. Event / Ops Layer

Purpose: make everything inspectable and recoverable

it includes

- task flow/ids log
- event log
- audit log
- change log
- versioning
- migration records
- rollback records
- failure reports
- recovery procedures
- OS self-tests

8. Design System Layer

Purpose: give the OS a real operating identity

it includes

- visual metaphor
- layout grid
- component contracts
- status states
- ink states
- approval states
- warning states
- failure states
- quarantine states
- recovery states
- dashboard panel rules

Final Folder Layout

NRG-Agent-OS/



README.md

PROJECT.md

CURRENT_STATE.md

NEXT_ACTION.md

DECISIONS.md

ISSUES.md

PATHS.md

CHANGELOG.md

os/

kernel/

boot.sequence.md

task.lifecycle.md

definition-of-done.md

operating-modes.md

capability-registry.schema.json

agent-contract.schema.json

task-card.schema.json

approval.schema.json

evidence.schema.json

router/

router.policy.md

intent-routing.policy.md

model-routing.policy.md

agent-routing.policy.md

skill-routing.policy.md

memory-routing.policy.md

risk-routing.policy.md

fallback-routing.policy.md

router.schema.json

permissions/

permission.policy.md

destructive-actions.policy.md

file-safety.policy.md

secrets-redaction.policy.md

approval-gates.policy.md

permission.schema.json

validation/

validation.policy.md

claim-verification.policy.md

path-verification.policy.md

diff-verification.policy.md

test-evidence.policy.md

drift-detection.policy.md

hallucination-detection.policy.md

```
no-scope-creep.policy.md
self-tests.md
validation.schema.json

review/
  fable-review-gate.md
  review-checklist.md
  acceptance-criteria.md
  quarantine-review.md

apps/
  dashboard/
    README.md
    design-system.md
    panels/
    components/
    routes/
    state/
    api-client/

  api/
    README.md
    routes/
    controllers/
    services/
    middleware/
    schemas/

runtime/
  hermes/
    hermes-runtime-contract.md
    hermes-learning-policy.md
    hermes-session-summary.schema.json
    hermes-memory-proposal.schema.json
    hermes-skill-proposal.schema.json
    hermes-automation-proposal.schema.json

workers/
  worker-contract.md
  task-card-template.md
  worker-result-template.md
  codex/
  gpt/
  gemini/
  local/

model-gateway/
  gateway.policy.md
  models.registry.json
  model-capabilities.schema.json
  fallback.policy.md
  benchmarks/
```

```
adapters/

memory/
  README.md
  memory.policy.md
  trust-levels.md
  promotion.policy.md
  indexes/
  persistent/
    decisions/
    project-state/
    paths/
    known-failures/
    lessons/
    skills/
    automations/
    validations/
    source-summaries/
  scratch/
    sessions/
    temp-context/
  quarantine/
    untrusted/
    stale/
    conflicting/
    rejected/

skills/
  README.md
  skill-registry.schema.json
  registry/
    filesystem.readonly-inventory.md
    project.scan-hooks-crons-skills.md
    coding.verify-imports.md
  proposed/
  deprecated/
  tests/

automations/
  automation.policy.md
  hook.policy.md
  cron.policy.md
  event-trigger.policy.md
  approval-gates.md
  rollback.policy.md
  registry/

validation/
  verify-paths.md
  verify-diff.md
  verify-claims.md
  verify-tests.md
```


- evidence rules
- quarantine rules
- OS self-tests
- event audit log
- versioning and migration policy
- secret redaction policy
- File review gate

The kernel is the armour. Without it, Hermes learns from chaos, workers improvise, and the dashboard becomes theatre.

Boot Sequence

On startup, NRS Agent OS must:

- load OS charter
- load permission policy
- load operating mode
- load path registry
- load model registry
- load agent contracts
- load skill registry
- load automation registry
- load memory tract policy
- scan quarantine state
- verify secrets are not exposed
- run OS self-test
- start event logger
- start API
- start dashboard
- start Hermes only after policies are loaded

If boot checks fail, the OS starts in read-only degraded mode.

Task Lifecycle

Every task moves through fixed states:

- intake
- classify intent
- estimate risk
- select operating mode
- check permissions

select model/agent
select skill
create task/sent
execute or dry-run
capture evidence
validate result
detect drift
detect hallucination
write event log
propose memory update
table review if required
accept, revise, reject, or quarantine
final report

No task can skip validation. No task can claim success without evidence.

Operating Modes

Readonly mode: inspect, search, report. Default.

Plan mode: propose actions, no file changes.

Dry-run mode: simulate changes and show intended diff or commands.

Sandbox mode: allow changes only inside a controlled sandbox path.

Patch mode: allow limited edits to approved files only.

Automation-proposal mode: draft hooks or new automations without enabling them.

Live-approved mode: execute approved actions with explicit user approval and rollback path.

Recovery mode: inspect damage, stop running agents, quarantine suspect changes, restore from known-good state.

No mode grants agent permission to delete, move, overwrite, mass edit, re-structure, install, migrate, reset, opt, edit hooks, or edit users.

Permission Policy

Default permission is readonly.

Blocked unless explicitly approved.

move
delete
overwrite
mass edit
install
migration
git reset
folder restructure
hook edit
git edit
destructive automation
secret exposure
permission policy edit
router policy edit
trusted memory promotion
core prompt modification

Every risky action requires

exact path
reason
risk level
expected change
rollback plan
approval record
evidence after action

Persistent Memory Architecture

Memory has three main zones.

Persistent memory stores approved, useful, long-term project truth.

Search memory stores session context and temporary work.

Quarantine memory stores untrusted, stale, conflicting, rejected, or suspicious material.

Trust levels:

verified
user confirmed
role enforced
agent enforced
static

untrusted

quarantined

No untrusted memory becomes active without validation

Memory copies

project state

current state

next action

decisions

issues

path registry

known failures

lessons

decision summaries

source summaries

skill history

automation history

validation history

Hannes multiple notes

Memory promotion requires

source reference

trust level

validation method

conflict check

reason for promotion

rollback path

event log entry

review status

Memory is context, not authority

Router Architecture

Every request passes through the router stack

input router

policy router

admin/router/tda

memory router

skill router

model:router
validation:router
fallback:router

Routing output must state:

task type
risk level
mode
chosen model
chosen agent
chosen skill
files allowed
files forbidden
commands allowed
commands forbidden
validation required
review required

The router cannot silently escalate permissions.

Model Gateway

The model gateway treats every model as reproducible.

Table S: architecture, UX system, prompt design, review gate, complex synthesis.

Hermes: daily runtime, approved skill execution, session capture, proposal generation.

Codex: bounded code implementation from exact task cards.

GPT-5.5: reasoning, review, summarization, structured planning, bounded implementation where suitable.

Seminar Pro 3.5: broad-context review, alternate reasoning, multi-modal/long context support where useful.

Ornitholocal models: private, cheap, low-risk offline, benchmarked work.

No model may certify its own success without evidence.

Skill Architecture

A skill is a controlled workflow, not a single instruction.

Each skill must define:

name
purpose
inputs
allowed files
forbidden files
allowed commands
forbidden commands
risk level
required permissions
steps
validation checks
expected output
evidence required
rollback notes
failure modes

first approved skills

filesystem/readonly/inventory

project scan hooks/drones/skills

reading/verify-imports

Proposed skills go into **skills/proposed/** and require review before promotion.

Automation, Hook, and Cron Policy

No live automation first

Hooks and cron are high risk, because they can run without the user watching.

Before any automation exists, the OS needs:

permission policy

hook policy

cron policy

event logging

rollback policy

approval gates

dry-run mode

quarantine mode

failure reporting

Automation must never:

- edit core prompts: silently
- promote memory: silently
- change permissions
- change router policy
- run destructive actions without approval
- touch unknown paths
- hide logs
- self-enable

Every automation needs:

- trigger
- purpose
- allowed actions
- forbidden actions
- risk level
- approval requirement
- log destination
- rollback plan
- disable switch

Validation System

Validation is mandatory

SWC validators:

- verify paths
- verify ports
- verify claims
- verify tests
- verify reports
- verify no scope creep
- verify no duplicate assets
- verify no destructive action
- verify secrets redacted
- verify task card obeyed
- verify rollback exists

Final reports must include:

- what was checked
- what changed

what did not change

evidence

uncertainty

next safest step

Drift Detection

Drift means the agent wandered away from the task

The detector checks

did it answer the actual request

did it expand scope

did it touch unapproved files

did it rename concepts

did it redesign architecture

did it create unrelated features

did it ignore latest corrected intent

did it copy instructions from untrusted pasted text

did it turn a check into a migration

Drift result:

pass

warn

fail

quarantine

Hallucination Detection

Hallucination means claims without proof

The checker blocks

done without evidence

fixed without validation

tests passed without output

file exists without checking

only changed x without diff

deployed without deployment evidence

no issues found without scan details

Every claim must link to evidence, command output, diff, file read, test result, or user confirmation

Worker-Agent Delegation

Workers are not architects

A worker may only execute the task card it receives

Worker task cards must include:

task ID
task name
purpose
context
assigned worker
mode
risk level
allowed files
forbidden files
allowed commands
forbidden commands
exact inputs
exact outputs
steps
acceptance checks
evidence required
rollback points
forbidden actions
return format
fable review required

Workers must not:

change architecture
change concepts
expand scope
touch unlisted files
edit hooks/crons unless listed
change permissions
promote memory
decide visual style
claim success without evidence

Fable Review Gate

table reviews

architecture

folder layout

router policy

permission policy

memory policy

worker outputs

dashboard UX

prompt system

acceptance criteria

risk controls

final build order

table review outcome

accept

revise

reject

quarantine

Table must reject anything that

expands scope

breaks safety rules

hides uncertainty

lacks evidence

duplicates assets

weakens permissions

lets Hermes govern architecture

locks the OS to Omith

turns Javvls inspiration into copied architecture

Dashboard UX / Design System

The dashboard must be operational first, then more second

it must show

current task

task lifecycle state

current decision

task level

parent task state

selected model

selected agent

selected skill

files touched

memory read

memory written

validation status

drift warnings

hallucination warnings

approval required

worker task cards

fable review status

Hermes runtime status

hook and cron status

quarantine items

recovery options

event log

Creative direction: avoid generic glowing node galaxies, neural brains, orbiting particles (the soft panels and pretty widgets with no operating value).

Better visual metaphors for Fable to choose from:

command reactor

agent traffic control

mission control map

neural workshop

forge/foundry

control board/OS

control tower

The winning metaphor should make failures visible instead of hiding them.

Hermes Runtime Integration

Hermes is the daily operator inside the OS.

hermes.mai

approve approved skills

rollback router and permission files

capture session summaries

propose lessons

propose memory updates
propose skill improvements
propose information candidates
report repeated failures

Hermes may not

define architecture
rewrite router policy
change permissions
silently change prompts
silently promote memory
edit hooks
edit cron
run destructive actions
update its own evolution

Hermes learning loop

observe
capture
draft proposal
validate
quarantine if uncertain
review
promote if approved
monitor
rollback if worse

Hermes should get smarter because the CS gives it clear rails, not because it is allowed to mutate the system freely.

Ornith / Local Model Strategy

Ornith is optional:

it must be

hand-manageable
swappable
disableable
upgradable
replaceable

Use local models for:
cheap summaries
private notes
intent classification
low-latency drafting
offline assistance
simple routing
repeatable low-latency workflows

Do not use local models for:
unverified success claims
destructive actions
permission changes
router changes
architecture ownership
complex in-app edits without review

JARVIS / Video Inspiration Handling

Use JARVIS for feel and interaction ideas only.

Valid extraction:
note-grounded answering
source-proof
fly-to-source concept
voice interaction
memory capture
model hot-swap UX
action confirmation gate
short personality flavor
cinematic motion language

Invalid extraction:
copying the memory gallery
making the graph the OS
using aesthetics as architecture
putting API keys in chat
hiding failures behind visuals
letting a demo sequence decide the build order

The Jarvis PDF is the cockpit inspiration. Mya Agent OS still needs the cockpit.

Self-Evolution Governance

Allowed as proposals:

- skill improvements
- prompt improvements
- tool descriptions
- workflow optimisations
- memory lessons
- automation improvements
- benchmarks
- comparison reports

Forbidden without approval:

- editing live system prompts
- changing permission policy
- changing router policy
- promoting trusted memory
- editing books
- editing cron
- running destructive automations
- rewriting core OS structure
- silently replacing skills
- claiming improvement without tests

Every evolution proposal must answer:

- what changed
- why it changed
- evidence of improvement
- tests passed
- what got worse
- risks remaining
- rollback method
- accept/reject/revert or quarantine

Event / Audit Log

Every run creates an event record

eventID
timestamp
user request
task ID
mode
risk level
router decision
model selected
agent selected
skill selected
files read
files changed
commands run
memory read
memory proposed
validation results
approval records
review result
final status
rollback notes

Logs must be readable from the dashboard and stored in reports.

OS Self-Tests

Before live work, the OS must test:

permission policy loads
readonly default active
path registry readable
model registry readable
memory trust policy active
quarantine folder exists
event logger works
worker task schema validates
review gate exists
secret revocation works
destructive actions blocked
dashboard can show failure status
if self-test fail, OS stays read-only

Secrets and Redaction Policy

Never paste or print:

- API keys
- tokens
- env contents
- credentials
- SSH keys
- private config values
- session cookies
- provider secrets

Agents may report secret locations, not secret values.

Example allowable report:

```
Secret-like value detected in .env. Value redacted.
```

Example forbidden report:

printing the actual key:

Versioning and Migration Policy

Every major OS change needs:

- version number
- change summary
- files touched
- reason
- risk level
- migration notes
- rollback plan
- validation results
- final review result

Migrations default to dry-run first

No migration may restructure folders without explicit approval

Recovery Policy

Recovery mode starts when

worker drifts
files are overwritten
hooks/scripts misbehave
memory is polluted
secrets are evinced
tests fail after changes
agent claims success without evidence
dashboard hides failure
router/permission policy breaks

Recovery steps

stop active agents
disable live automations
switch to read-only
snapshot current state
identify changed files
quarantine suspect memory
inspect event logs
compare dfts
restore known-good files
run self-tests
produce recovery report
require approval before returning to live mode

First Build Order

Build the boring core first:

Phase 1: Source of truth files

PROJECT.md

CURRENT_STATE.md

NEXT_ACTION.md

DECISIONS.md

ISSUES.md

PATHS.md

AGENT RULES.md

PERMISSION_POLICY.md

MODEL_ROUTING_POLICY.md

VALIDATION_POLICY.md

HERNIES_RUNTIME_POLICY.md

concept designs



lock-schemas

lock-it-sqls

definition of done

operating modes

sql-read-schema

agent contact schema

evidence schema

event log schema

Phase 3: Model and permissions

intent router

task router

permission router

model router

skill router

memory router

fallback router

Phase 4: Memory system

persistent memory

short-term memory

quantum memory

multi-level

in-memory rules

session summaries

source summaries

Phase 5: Tool skills

file-system-read-only-inventory

project-search-backs-for-gps-skills

coding-venis-imports

Phase 6: Final validation

verify paths

verify claims

verify import scope/resp

verify diff

verify tests

Phase 7: Hermes runtime capture

session summaries

lesson proposals

skill proposals

memory proposals

failure reports

Phase 8: Automation policy

hook policy

cron policy

approval gates

rollback plans

Phase 9: Dashboard

only after kernel, router, permissions, memory, validation, and event logs exist

Acceptance Criteria

NRG Agent OS v0.1 is acceptable only when

kernel docs exist

permission policy exists

readonly default exists

task lifecycle exists

definition of done exists

memory trust model exists

quarantine policy exists

router policy exists

agent contracts exist

worker task card template exists

table review gate exists

validation policy exists

info detection exists

notification detection exists

secrets redaction policy exists

event audit log schema exists

OS self tests exist

hermes runtime contract exists

Omni-local strategy exists
hook/cron policy exists
dashboard design system exists
build order exists
recovery policy exists
what-not-to-build-ver-1st exists

Risks and Controls

Major risks

workers redesign the OS
Hermes becomes uncontrolled authority
Omni becomes lock-in
dashboard gets built before kernel
memory graph becomes the OS
hooks/cron run before safety exists
agents mint paths
agents overwrite or duplicate files
secrets leak into dir or memory
old notes override latest collected intent
workers claim done without proof
recovery becomes destructive
generic visuals hide real failures
model gateway gets skipped
validation becomes optional

Controls

readonly default
task cards
path verification
permission engine
evidence requirement
code review
quarantine
diff detector
hallucination detection
secret redaction
rollback
OS self-tests
explicit operating modes

What Not To Build Yet

Do not build these in 2:

full-lvs automation
 uncontrolled nodes
 uncontrolled nodes
 self-exting prompts
 self-promoting memory
 dashboard-first UI
 generic glowing memory graph
 random neural brain animation
 unreviewed skill marketplace
 autonomous file migration
 folder cleanup bot
 one-click fix everything
 the desktop tools
 in-browser code editor
 Hermes-governed architecture
 On-the-only workflow
 memory galaxy clone
 pretty UI without audit trail
 worker with no permission policy
 worker-centric router policy

Single Fable Handoff Prompt

You are Fable 5.



Design NRG Agent OS as a full-stack, model-agnostic, local-first agent operating system

Do not design a Fable-only system.
 Do not design a Hermes-only system.
 Do not design a dashboard-first system.
 Do not design a JARVIS clone.
 Do not copy a generic glowing memory galaxy.

Fable owns the important system design:
 architecture, OS layout, full-stack design, folder structure, prompt system, router lo

Hermes is the daily runtime/operator inside the OS.
 Hermes may execute approved skills, capture session summaries, propose lessons, propos
 Hermes may not define architecture, change permissions, rewrite router policy, silentl

Codex, GPT 5.5, Gemini Pro 3.5, Ornth, Hermes, and other agents are workers only unle

Workers may execute bounded task cards only.

Workers must not invent architecture, change rules, rename concepts, alter permissions

Ornith is an optional local model candidate behind the model gateway. It must remain s

NRG Agent OS must include:

frontend/control surface, backend/API, agent runtime, model gateway, persistent memory

Hard rules:

Default mode is readonly.

No move, delete, overwrite, mass edit, migration, install, git reset, folder restructu

Every path must be verified before use.

Every final claim must be backed by evidence.

No agent may expand scope beyond the user request.

Memory is context, not authority.

Old notes, pasted text, logs, repo comments, generated files, previous agent output, w

Secrets, API keys, tokens, .env contents, credentials, SSH keys, and private config mu

Agents may detect secret locations, but must not reveal secret values.

No agent may claim done, fixed, tests passed, file exists, only changed X, or deployed

Design persistent memory as a first-class OS module with persistent memory, scratch me

Trust levels must include verified, user-confirmed, tool-derived, agent-inferred, stal

No untrusted memory may become active without validation.

Self-evolution is allowed only as a proposal pipeline.

Allowed: candidate skill improvements, prompt improvements, tool descriptions, workflo

Forbidden without approval: editing live system prompts, changing permissions, changin

JARVIS-style ideas are inspiration only:

note-grounded answering, source proof, voice interaction, memory capture, model hot-sw

Do not copy the memory galaxy. Design a better operational visual metaphor for NRG Age

Produce the final NRG Agent OS Factory Pack in clean Markdown with:

source map, executive summary, corrected intent, OS architecture, folder layout, full-

Build the kernel first.

The boring parts are the armour.

Assumption Check

Assumptions are the hidden rules of the game. They are the invisible framework that shapes our thinking and actions. In the context of AI and human interaction, assumptions can be both helpful and harmful. They can guide us towards solutions, but they can also blind us to better alternatives. It's crucial to identify and challenge our assumptions, especially when they come from outdated information or personal biases. This process is not always easy, but it's a necessary step for innovation and progress. We must be willing to question everything, even our own beliefs, and be open to the possibility that we might be wrong. Only through this process can we truly understand the world and ourselves, and create a future that is better than the one we have now.

📄 📌 🔄 ... 📖 Sources

